
Examen
Algorithmique et programmation en python
Durée : 1H30

Exercice 1

6pts

Soit une suite de clés dans une liste L_N de N entiers tous différents et appartenant à l'intervalle $[1 .. N]$. Par exemple pour $N=8$, $L_8 = [5, 8, 2, 6, 3, 4, 1, 7]$

1. Ecrire la fonction **verifier** qui vérifie que les éléments d'une liste L_N appartiennent à l'intervalle $[1 .. N]$ et sont tous différents.
2. Evaluer en ordre de grandeur, le coût temporel de cette fonction

Exercice 2

14pts

L'évaluation d'expressions arithmétiques est essentielle dans le processus de compilation et d'interprétation des programmes. Lorsque des expressions infixées sont rencontrées, elles sont souvent converties en expressions postfixes (opérateurs suivant les opérandes) pour simplifier l'évaluation. Une expression arithmétique infixe est un texte composé d'opérateurs (+, -, * et /), d'opérandes (constantes entières et/ou réelles) et éventuellement des parenthèses (pour forcer des priorités). Dans ce cadre, on se propose d'explorer cette conversion et d'implémenter l'évaluation d'expression postfixe. Cette conversion est possible grâce à l'utilisation de la structure de pile (LIFO) comme structure auxiliaire pour gérer les opérateurs pendant la conversion. Exemple d'expression arithmétique infixée : "3.14 + 5 * (2.718 - 8)" son équivalent postfixe est "3.14 5 2.718 8 - * +"

1. /3pts Implémenter une fonction **tokenize(expression)** qui prend une expression arithmétique infixée en entrée et renvoie une liste de tokens (opérandes et opérateurs) sans utiliser d'expressions régulières. Utiliser une boucle pour parcourir la chaîne de caractères et construire la liste de manière itérative.
tokenize("3.14 + 5 * (2.718 - 8)") doit produire : ['3.14', '+', '5', '*', '(', '2.718', '-', '8', ')']
2. /4pts Implémenter une classe **Pile** avec des méthodes **empiler**, **depiler**, **getSommet** et **estVide**. Cette pile sera utilisée pour gérer les opérateurs lors de la conversion de l'infixe en postfixe.
3. /4pts Implémenter une fonction **infixeToPostfixe(expression_infixe)** qui prend une expression arithmétique infixée en entrée et renvoie l'expression postfixée correspondante sous la forme d'une liste en suivant les étapes suivantes :
 - a. Initialiser une liste de sortie postfixe.
 - b. Initialiser une pile pour les opérateurs.
 - c. Parcourir chaque token de la liste obtenue après la tokenization et distinguer les cas suivants :
 - d. Si le token est un opérande (entier ou réel), ajouter le directement à la liste de sortie postfixe.

- e. Si le token est un opérateur, dépiler tous les opérateurs de la pile ayant une priorité supérieure ou égale à celui-ci, et ajouter les dans l'ordre à la liste de sortie postfixe. Empiler le nouvel opérateur sur la pile.
 - f. Si le token est une parenthèse ouvrante, empiler la directement.
 - g. Si le token est une parenthèse fermante, dépiler les opérateurs jusqu'à trouver la parenthèse ouvrante correspondante et ajouter les à la liste de sortie postfixe.
 - h. Une fois le parcours des tokens terminé, dépiler tous les opérateurs restants de la pile et ajouter les à la liste de sortie postfixe.
4. /2pts Implémenter une fonction `evaluaerPostfixe (liste_postfixe)` qui prend une liste expression postfixe en entrée et qui renvoie le résultat de l'évaluation.
5. /1pts Donner un code de test pour tester votre implémentation en utilisant une expression arithmétique infixée avec des réels.